

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification
for Online Bookstore – Team 15

Item	Student Entry
Project Title	Online Bookstore – Web-based Book Ordering & Catalog Management System
Project ID	BOOKSTORE-2026-T15
Group Number	Team 15
Team Members	Anushka P – PES1202401127 / B S Keertana – PES1202400463 / Bhagath Naveen Mullakara – PES1202403272 / Bhuvan Raj H – PES1202502988
Course	UE24CS341A – Software Engineering
Technology Stack	MERN – MongoDB, Express.js, React, Node.js
Development Methodology	Agile
Version	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Online Bookstore, a web-based application that lets a bookstore list its catalog online and lets customers browse, search, and purchase books. This SRS covers the full system: the customer-facing storefront and the Administrator / Staff dashboard.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team (all four members), the course instructor/evaluator, and anyone reviewing the project's requirements baseline. Section 2 gives context and scope; Section 3 covers interfaces; Section 4 covers use cases and diagrams; Section 5 lists detailed functional requirements; Sections 6–7 and the appendices cover non-functional, security, and supporting reference material.

1.3 Product Scope

Online Bookstore digitizes catalog browsing and order management for a small independent or campus bookstore. It replaces in-person, walk-in ordering with a self-service storefront for customers and a lightweight management dashboard for Administrator / Staff, without requiring a costly, over-engineered e-commerce platform.

1.4 References

- Team 15 Synopsis / Project Proposal (Online Bookstore), v1.0
- PES University – Jackfruit Software Engineering Project Guidelines, UE24CS341A
- PES University – Software Engineering Level 3 Student Submission Guide & Exemplar

2. Overall Description

2.1 Product Perspective

Online Bookstore is a new, self-contained web application; it is not a follow-on to an existing system. It consists of a React single-page frontend, a Node.js/Express REST API backend, and a MongoDB database, communicating over HTTPS.

2.2 Product Functions

- Customer account registration and login
- Catalog browsing, search, and book detail viewing
- Cart management, checkout, and order placement
- Order status tracking and order history
- Administrator / Staff catalog management (add/edit/disable books, update stock)
- Administrator / Staff order management (view orders, update fulfillment status)

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Customer	Browse, search, order, complete mock payment, and track own orders	Non-technical, occasional use
Administrator / Staff	Manage catalog and orders	Higher privilege; store staff

2.4 Operating Environment

The application runs as a responsive web app accessible from modern desktop and mobile browsers (Chrome, Firefox, Edge). The backend runs on Node.js and connects to a MongoDB instance (local or MongoDB Atlas).

2.5 Design and Implementation Constraints

- Technology stack is fixed to MERN (MongoDB, Express.js, React, Node.js) — the team's chosen stack over the alternative Django option
- Agile methodology is mandatory for this course; work is organized into sprints with a visible backlog
- Git/GitHub must be used with real, incremental commit history from every member
- CI/CD pipeline, code reviews, and security validation are mandatory course deliverables
- Authentication must use JWT-based sessions

2.6 Assumptions and Dependencies

- A real payment gateway is out of scope; a mock/simulated payment service is assumed acceptable
- The catalog will be seeded with sample book data for development and testing
- A stable internet connection and a MongoDB instance (local or Atlas) are available to all team members during development

3. External Interface Requirements

3.1 User Interfaces

The customer storefront includes: home/catalog page, search results, book detail page, cart, checkout, and order history. The Administrator / Staff dashboard includes: catalog management and order management views. Both are responsive and built with React.

3.2 Software Interfaces

- Frontend (React) communicates with the backend via a REST API (JSON over HTTPS)
- Backend (Node.js/Express) communicates with MongoDB via the Mongoose ODM
- Backend communicates with a mock/simulated Payment Service for order payment authorization

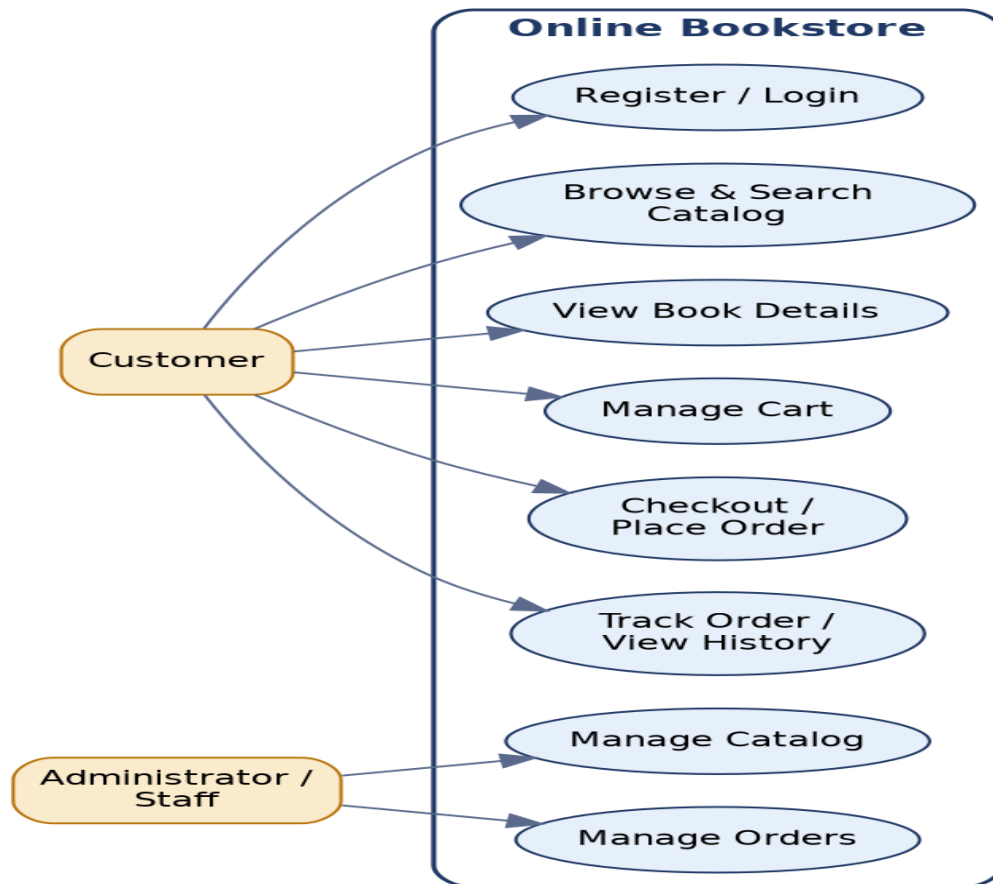
3.3 Communications Interfaces

All client-server communication uses HTTPS. Authenticated requests carry a JWT bearer token in the Authorization header. Data is exchanged as JSON.

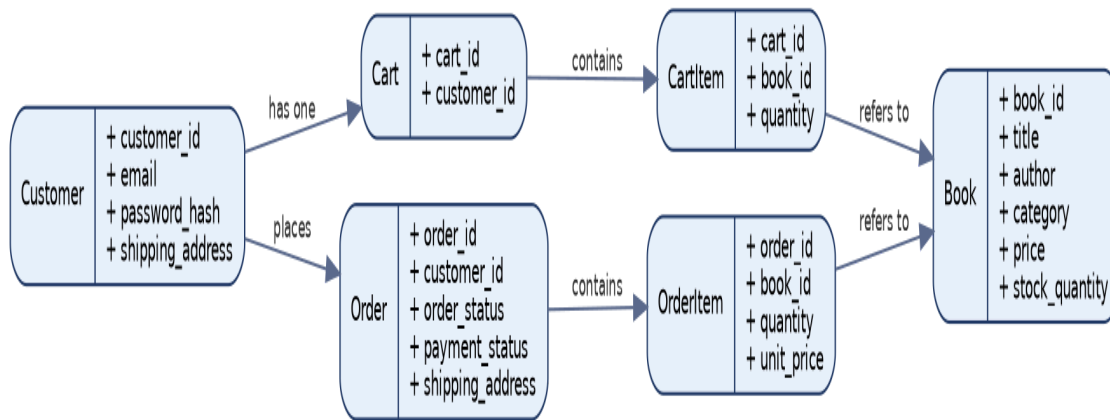
4. Analysis Models — Use Cases and Diagrams

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Customer	Application available	Authenticated session
UC-02	Browse & Search Catalog	Customer	None	Matching books displayed
UC-03	View Book Details	Customer	Book exists	Book detail page shown
UC-04	Manage Cart	Customer	Logged in	Cart updated
UC-05	Checkout / Place Order	Customer	Logged in; valid cart	Order created
UC-06	Track Order / View History	Customer	Order exists	Current status displayed
UC-07	Manage Catalog	Administrator / Staff	Administrator/Staff authenticated	Catalog updated
UC-08	Manage Orders	Administrator / Staff	Administrator/Staff authenticated	Order status updated

4.1 Use Case Diagram



4.2 Domain / Data Model Diagram



The domain model above covers the Customer, Cart, CartItem, Book, Order, and OrderItem entities and their relationships, matching the field layout in Appendix B.

5. System Features — Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a customer to register an account using a unique email address and a password.	High	Functional test
FR-002	The system shall allow a registered customer to log in using their email and password.	High	Functional test
FR-003	The system shall allow a customer to browse the book catalog without requiring login.	High	Functional test
FR-004	The system shall allow a customer to search the catalog by title, author, or category.	High	Functional test
FR-005	The system shall display a book detail page showing title, author, category, price, stock status, description, and cover image.	High	Functional test
FR-006	The system shall allow a logged-in customer to add an in-stock book to their cart.	High	Functional test
FR-007	The system shall allow a customer to update the quantity of, or remove, an item in their cart.	High	Functional test
FR-008	The system shall calculate the cart total from item price and quantity for all items in the cart.	High	Unit + system test
FR-009	The system shall allow a customer to place an order only after all mandatory checkout information (shipping address, cart contents) is valid.	High	Negative + positive test
FR-010	The system shall assign a unique order number to every successfully created order.	High	Integration test
FR-011	The system shall display order status as PLACED, PROCESSING, SHIPPED, DELIVERED, or CANCELLED.	High	System test
FR-012	The system shall allow a customer to view their own order history and current order status.	High	Functional test
FR-013	The system shall allow Administrator / Staff to add, edit, or disable book listings in the catalog.	Medium	Functional test
FR-014	The system shall allow Administrator / Staff to update stock quantity for a book.	Medium	Functional test
FR-015	The system shall allow Administrator / Staff to view incoming orders and update their fulfillment status.	High	Functional test
FR-016	The system shall record payment success or failure against the corresponding order using a	High	Integration test

	mock/simulated payment service and shall not confirm an order when payment fails.		
--	---	--	--

6. Other Nonfunctional Requirements

6.1–6.2 Performance, Safety & Availability

Safety: Online Bookstore is a standard e-commerce web application with no physical, medical, or life-safety implications; no safety-critical requirements apply beyond the standard security requirements in Section 6.3.

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of catalog search requests shall receive a response within 1.5 seconds under a load of 50 concurrent users.	Performance test
NFR-002	Availability	The application shall be available at least 99% during the defined operating window, excluding planned maintenance.	Operational review
NFR-003	Usability	A new customer shall be able to place a basic order in no more than 5 primary screens.	Usability walkthrough
NFR-004	Reliability	An order shall not be created twice when a customer retries checkout after a payment timeout.	Negative/integration test
NFR-005	Maintainability	Business logic shall be separated into identifiable service modules with automated unit tests.	Code review
NFR-006	Testability	Core services shall expose testable interfaces without requiring the production payment service.	Unit/integration test

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall not be stored in plaintext; they shall be hashed before storage.	Code/configuration review
SEC-002	Role-based access shall prevent customers from accessing Administrator/Staff-only endpoints.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied fields before processing.	Negative/security tests
SEC-004	Authenticated APIs shall reject expired or invalid JWT tokens.	Security test
SEC-005	Sensitive payment information shall not be stored unnecessarily by the application.	Design review
SEC-006	Security-relevant events such as failed login and unauthorized access attempts shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability is prioritized over raw performance for this course project, given the short timeline
- Usability: checkout must be completable without prior training or instructions

6.5 Business Rules / Domain Rules

- Only in-stock books can be added to a cart or order
- A customer may cancel or modify an order only while its status is PLACED
- Only Administrator / Staff can add, edit, or disable catalog items
- Only Administrator / Staff can update an order's fulfillment status
- An order must contain at least one valid item

- A mock payment failure shall not result in a confirmed order

7. Other Requirements

No additional database, legal, or localization requirements beyond those already listed are identified at this stage.

Appendix A: Glossary

- Cart — a customer's temporary, unordered selection of books before checkout
- Catalog — the full set of books available for browsing and purchase
- Checkout — the process of converting a cart into a confirmed order
- RBAC — Role-Based Access Control, restricting actions by user role (Customer vs. Administrator / Staff)
- JWT — JSON Web Token, used to authenticate API requests
- Stock / Inventory — the quantity of a given book currently available for sale
- Category — a book classification used for browsing and filtering the catalog (e.g., Fiction, Non-fiction, Academic)
- Mock/Simulated Payment — a stand-in payment step that returns a success or failure result without contacting a real payment gateway

Appendix B: Field Layouts

Field	Length	Data Type	Description	Mandatory
customer_id	36	ObjectId/String	Unique customer identifier	Y
shipping_address	300	String	Customer delivery address	Y
email	100	String	Unique login email	Y
password_hash	60	String	Hashed customer password	Y
book_id	36	ObjectId/String	Unique book identifier	Y
title	150	String	Book title	Y
author	100	String	Book author	Y
category	50	String	Book category/genre (e.g., Fiction, Academic)	Y
price	10,2	Decimal	Current selling price	Y
stock_quantity	6	Integer	Units currently in stock	Y
order_id	36	ObjectId/String	Unique order identifier	Y
order_status	20	Enum/String	PLACED/PROCESSING/SHIPPED /DELIVERED/CANCELLED	Y
payment_status	20	Enum/String	Mock payment result	Y

Appendix C: Requirement Traceability Matrix (Initial)

Architecture, design, and code references will be completed once Deliverable 3 (Design) is finalized; this initial matrix records requirement IDs against planned test coverage.

Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
FR-001	Customer registration	TBD	TBD	TBD	TBD	TBD
FR-006	Add to cart	TBD	TBD	TBD	TBD	TBD
FR-009	Place order	TBD	TBD	TBD	TBD	TBD
SEC-002	Role-based access	TBD	TBD	TBD	TBD	TBD
NFR-001	Search performance	TBD	TBD	TBD	TBD	TBD

Part B: Initial Validation Specification

At this requirements stage, validation planning is focused on how each requirement category will eventually be validated; this is not the final test report.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Valid/invalid checkout, cart edits, search filters
Security	Authentication/authorization + security testing	Customer attempts an Administrator/Staff-only endpoint
Performance	Load/response-time test	Concurrent catalog search and checkout requests
Usability	Task-based walkthrough	New customer places their first order
Reliability	Failure/retry scenarios	Payment timeout and retry does not duplicate order
Unit	Automated unit tests + coverage	Cart total calculation, stock validation

Part C: Project Plan

Work Breakdown Structure

Work Package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	All (led by Keertana & Bhuvan)	4	Architecture/Design
WP3	Catalog & search module	Anushka P	6	Working feature
WP4	Cart & checkout module	B S Keertana	6	Working feature
WP5	Auth module (registration/login)	Bhagath Naveen Mullakara	5	Working feature
WP6	Admin catalog & order management module	Bhuvan Raj H	7	Working feature
WP7	Testing + CI/CD + security	All	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

Tool Stack

Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Frontend	React (Create React App / Vite)	Component source, build output
Backend	Node.js + Express	API source, route documentation
Database	MongoDB (via Mongoose ODM)	Schema definitions, seed data
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline run logs / screenshots
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest + Supertest (backend), React Testing Library (frontend)	Test reports
Static analysis	SonarQube or similar / ESLint	Analysis report
API testing	Postman	Collection/results
Design	Draw.io / PlantUML	UML diagrams

Agile lifecycle justification: the team will work in six sprints (see the synopsis's short-term plan), each producing a demonstrable increment, with a GitHub Projects/Jira backlog of user stories mapped to the functional requirements above.